

CYNA DEV

Rapport de Test

Module Sécurité Applicative

Projet : CYNA DEV

Périmètre : Injection SQL, XSS, en-têtes de sécurité HTTP, CORS, exposition de fichiers sensibles

Résultat global : 6/6 tests conformes

1. Objectif

Ce rapport documente des tests de sécurité applicative réels (tentatives d'attaque effectivement exécutées contre l'API en fonctionnement, pas une simple revue de code) : injection SQL, injection XSS, en-têtes HTTP de sécurité, politique CORS et exposition de fichiers sensibles.

2. Environnement de test

Tests exécutés contre le backend réel connecté à PostgreSQL, sans aucune modification du code source. Chaque tentative d'attaque est une requête HTTP réelle envoyée à l'API, dont la réponse et l'état de la base de données ont été vérifiés après coup.

3. Résultats détaillés

ID	Fonctionnalité testée	Comportement attendu	Résultat obtenu (réel)	Statut
T6.1	Injection SQL dans le paramètre de recherche (?q=)	Aucune erreur SQL, requête neutralisée, table intacte	200, 0 résultat, aucune erreur serveur ; vérification en base : table Utilisateurs toujours intacte (7 lignes) — requêtes paramétrées (pg) confirmées efficaces	OK
T6.1bis	Injection SQL dans le formulaire de connexion (bypass tenté)	Aucune connexion obtenue	400 — rejeté dès la validation du format d'email (express-validator), avant même d'atteindre la requête SQL	OK
T6.2	Injection XSS dans le formulaire de contact (<script>)	Le payload ne doit pas s'exécuter côté navigateur au rendu	201 — le payload est stocké tel quel en base (l'API ne filtre pas les entrées) ; vérification du code frontend : aucun composant n'utilise dangerouslySetInnerHTML, donc React échappe automatiquement le contenu à l'affichage — pas d'exécution possible	OK
T6.3	En-têtes de sécurité HTTP (Helmet)	x-content-type-options=nosniff, CSP présente, pas de x-powered-by	Confirmé : x-content-type-options=nosniff, x-frame-options=SAMEORIGIN, Content-Security-Policy présente, Strict-Transport-Security présente, aucun en-tête x-powered-by (Express masqué)	OK
T6.4	Requête avec une origine CORS non autorisée	L'origine malveillante ne doit pas être reflétée	L'en-tête Access-Control-Allow-Origin renvoie uniquement l'origine autorisée configurée (FRONTEND_URL), jamais	OK

ID	Fonctionnalité testée	Comportement attendu	Résultat obtenu (réel)	Statut
			l'origine du client — pas de réflexion dynamique dangereuse	
T6.5	Tentative d'accès direct au fichier .env via HTTP	404 Not Found	404 — aucun fichier sensible exposé	OK

4. Synthèse

Indicateur	Valeur
Tests exécutés	6
Conformes (OK)	6
Non conformes (KO)	0
Taux de réussite	100 %

5. Constats notables et recommandation

- Aucune faille critique détectée sur les vecteurs testés (injection SQL, XSS, CORS, en-têtes HTTP, exposition de fichiers).
- Point de vigilance : l'API ne filtre/n'échappe pas les entrées utilisateur avant stockage (ex. formulaire de contact) ; la protection contre le XSS repose entièrement sur l'échappement automatique de React côté front. C'est une architecture valable et courante, mais elle signifie que toute évolution future du code qui introduirait un dangerouslySetInnerHTML sur du contenu utilisateur (ex. affichage enrichi des messages dans le back-office) recréerait immédiatement une faille XSS. Recommandation : ajouter une validation/échappement défensif côté API en complément, pour ne pas dépendre uniquement de la couche d'affichage.
- Le rate-limiting (module Authentification) constitue une protection efficace supplémentaire contre les attaques par force brute, déjà vérifiée dans le rapport correspondant.

6. Conclusion

6 scénarios sur 6 conformes. La sécurité applicative de base est solide. Une seule recommandation d'amélioration défensive est formulée (validation/échappement en profondeur côté API), sans faille exploitable identifiée à ce stade.